

Getting Started for System Administrators

DCGM gives the software on a GPU host a shared management plane. Instead of every monitor, scheduler, and troubleshooting tool maintaining its own view of the hardware, they can use one long-lived host engine for inventory, telemetry, health, and management operations.

Operate one site-managed DCGM endpoint per host unless the platform supplies DCGM through another supported management agent. Complete [Install DCGM](#) before using the examples below.

Why the Host Engine Is Long-Lived

The host engine is more than a command dispatcher. It owns the entity namespace, field watches, retained samples, reusable groups, module state, and job records. A short-lived `dcgmi` process can create or inspect that state without owning its lifetime, and several clients can consume the same collection stream.

This design matters on a managed node. A scheduler can define the GPUs in an allocation, a monitoring agent can retain environmental data, and an operator can inspect the same host without each client starting an independent collection loop. The engine must remain available for those clients to share state and history.

If a platform management agent already provides the DCGM endpoint, use its documented address and lifecycle. Starting another engine creates a separate inventory and cache rather than extending the first one.

Inventory Is the Engine's Current View

Ask the engine which entities it manages:

```
$ dcgmi discovery --list
```

An entity is any object to which DCGM can attach fields or operations, not just a physical GPU. Depending on the system, the inventory can include MIG instances, NVSwitches, NVLink ports, NVIDIA CPUs and cores, or ConnectX devices.

The numeric IDs are local to the running host engine. A GPU UUID or PCI bus ID is the appropriate bridge to scheduler and asset inventory:

```
$ dcgmi discovery --info a --gpuid 0
```

Discover IDs on the target host instead of copying them from another node or an earlier configuration. On a MIG-enabled system, also read the current parent-child hierarchy:

```
$ dcgmi discovery --compute-hierarchy
```

An entity group gives later operations a reusable selection, such as the GPUs assigned to one job. It does not reserve those GPUs or enforce access. The scheduler and operating system remain responsible for placement and isolation.

Telemetry Is Sampled State

DCGM fields describe inventory values, gauges, counters, and events. A watch tells the host engine which entity-field pairs to collect, how often to collect them, and how much history to retain.

List the field catalogue, then observe GPU temperature (field 150) and power usage (field 155):

```
$ dcgmi dmon --list
$ dcgmi dmon --field-id 150,155 --count 5
```

`dmon` creates temporary watches and prints retained samples for all GPUs by default. Reading a sample does not usually query the hardware again. Its timestamp says when DCGM collected it.

A field’s presence in the catalogue does not guarantee support on every entity or device generation. `N/A` can mean unsupported, unavailable, not permissioned, or not sampled yet. Check field scope, entity support, permissions, and sample timing before treating it as a hardware failure.

The shared cache is useful for operational history but is not a monitoring database. Bound retention, measure host-engine overhead with [Introspection](#), and export data that must survive a service restart.

From Observation to Management

Core services supply facts and selections. Feature modules interpret those facts or act on the selected hardware.

Operational questions		
Question	DCGM model	Continue with
What hardware does the engine manage?	Entities and identity	Entities and Groups

Question	DCGM model	Continue with
What changed during an interval?	Fields, watches, and counters	Fields and Watches
How are GPUs and links related?	Topology and link entities	Topology and NVLink
Did a process or scheduler job use the allocation as expected?	Process watches and job windows	Process and Job Statistics
Has passive monitoring found a problem?	Health watches over retained fields	Health Monitoring
Is the node ready to run a workload?	Active deployment and hardware tests	Diagnostics

Passive observation is normally safe while workloads run. Configuration, resets, and active diagnostics can change or stress shared hardware. Perform them through scheduler-controlled maintenance, prologue, or epilogue workflows rather than from an unrelated monitoring client.

The Engine Acts with Its Own Credentials

Running `sudo dcgmi` does not elevate an already-running host engine. The engine performs the device, file, and executable access required by the operation, so its credentials determine whether the operation succeeds.

Run the host engine as root for full functionality. Some inventory and telemetry can work with fewer privileges, but configuration, accounting setup, diagnostics, device files, and external executables can require additional access. A separate diagnostic service account can introduce a second credential boundary.

When an operation works under one service identity but not another, investigate the specific device, file, executable, or socket permission before broadening access.

The Service Is a Trust and Failure Boundary

A pathname-based Unix-domain socket is local to the host and is governed in part by filesystem permissions. A TCP listener can be reachable from other hosts, and DCGM does not add authentication, authorization, or encryption to its TCP protocol. Keep TCP access on a trusted management network. See [the host-engine listener security note](#) before changing the listener.

Entity groups, field groups, watches, cached samples, job records, and module state belong to one running engine. In this documentation, *persistent* can mean that an object survives the client that

created it; it does not mean the object is stored on disk. Recreate site-owned monitoring and job state after an engine restart.

Deleting a group removes the selection but does not reverse configuration already applied to its members. See [HOST-ENGINE STATE AND RESTARTS](#) for the complete restart boundary.

Next Steps

- [Customizing the systemd Service](#) explains how to customize the packaged host-engine service, apply changes safely, and troubleshoot effective systemd settings.
- [Configure Prometheus for DCGM Exporter](#) configures Prometheus to scrape and verify installed DCGM Exporter targets.
- [Core Services](#) covers the entity, field, watch, and lifetime model.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.